Smart Election Management & Voter Analytics System

Project Report

Case Study122: Smart Election Management & Voter Analytics System

Name:-Harish Malviya
Roll Number:-150096725162
College:-ITM Skills University,Kharghar
Batch:-2025-2029
Course:BTech (CSE)
Subject:-DSA-1

Project Title

Smart Election Management & Voter Analytics System

A console-based software solution developed using C++ and Data Structures to efficiently manage voter records, process registrations, handle election data recovery, optimize logistics, and securely verify voter identity.

Problem Statement

Introduction

Democratic processes handle millions of voter records and require immense logistical coordination. Managing voter registrations, preventing fraudulent voting, tracking polling station capacities, and establishing efficient transport routes for election materials are critical for conducting fair elections. The Smart Election Management & Voter Analytics System is designed to streamline election operations through optimized data handling and logistical route planning.

Objective

The objective is to manage regional and nationwide voter databases, securely verify voter identities, track polling station capacities, provide undo capabilities for administrative errors, and optimize election logistics routes using graph algorithms.

Industry Context

Government & Public Administration: Election commissions process diverse demographics and massive volumes of data. Efficient digital management improves transparency, ensures zero voter duplication.

Voter Registration Using Queue

New voter registration requests enter a Queue and are processed in a First-In-First-Out (FIFO) order, ensuring fairness and systematic administrative processing.

Data Recovery Using Stack

Administrative actions (like registering a voter or casting a vote) are tracked. A Stack data structure enables immediate "Undo" operations, popping the last action to recover the previous system state.

Secure Verification Using Hashing

Voter ID cards are hashed into a Hash Table (size 211). This allows for $O(1)$ average-time complexity when verifying a voter's identity before they cast a ballot, preventing double-voting.

Voter Database Using BST & AVL Trees
Processed voters are inserted into both a Binary Search Tree (BST) and a self-balancing AVL Tree. This ensures that the voter ledger is maintained securely, with the AVL tree providing guaranteed O(log N) search times.

Station Ranking Using Sorting
Polling stations are dynamically ranked based on voter volume using the Merge Sort algorithm, allowing authorities to prioritize resource allocation to high-density regions.

Deliverables
The system provides a digital election dashboard, an automated registration pipeline, an $O(1)$ verification system, logistic route calculations, and ranked polling station analytics.

Conclusion
The Smart Election Management system demonstrates how Queues, Stacks, Hash Tables, AVL Trees, Merge Sort, and Graph Algorithms can modernize government administration, enforce transparency, and optimize resource allocation during elections.

Objectives

The primary objectives of the project are:

- To process voter registration requests using Queue data structures.
- To provide secure, instant voter verification using Hash Tables.
- To manage election data recovery and undo operations using a Stack.
- To store voter databases efficiently using BST and AVL Trees.
- To rank polling stations by voter volume using Merge Sort.
- To retrieve voter information efficiently using Binary Search.
- To model election logistics and physical routes using Graphs.
- To optimize material dispatch planning using BFS, DFS, and Dijkstra's Algorithm.
- To display a comprehensive election dashboard with turnout statistics.

Design / Architecture

System Overview

The Smart Election Management System is designed as a modular, menu-driven C++ application that simulates the workflow of a centralized election commission. The architecture integrates multiple Data Structures and Algorithms to efficiently manage requests, secure the voting process, and analyze logistics.

Architectural Components

1. User Interaction Layer

The User Interaction Layer acts as the interface between election officers and the system. It provides menu-driven operations through which users can:

- Submit registration requests
- Process pending requests
- Undo administrative actions
- Search and verify voters
- Rank polling stations
- Analyze logistic routes
- View the central dashboard

---

2. Voter Management & Ledger Module

This module is responsible for maintaining all official voter records. Each voter

contains:

- Voter ID
- Name
- Age
- Gender
- Voter Card No
- Station ID
- Voting Status (hasVoted)

Voters are stored inside centralized tree structures which serve as the primary database of the system.

Responsibilities

- Register new voters
- Update voting status
- Store voter metadata

- Support searching and reporting operations

Data Structure Used

BST & AVL Trees

Reason for Selection

AVL Trees provide self-balancing structures that guarantee efficient storage and retrieval in O(log N) time, making them ideal for managing large, dynamic voter databases while preventing degradation in search performance.

---

3. Registration Queue Module

All user inputs are validated before being forwarded to the corresponding

processing modules.

Workflow

Request A Submitted
Request B Submitted
Request C Submitted

Processing Order:
A→ B → C

Responsibilities

- Maintain registration order
- Ensure fairness in processing
- Handle pending registrations

Data Structure Used

Queue

Reason for Selection

Voter registrations are naturally processed in the order they are received by election officers. Queue provides an efficient and realistic representation of this workflow.

---

4. Data Recovery Module

Certain administrative actions (like registering a voter or casting a vote) require the ability to be reversed in case of human error. Such operations are tracked and pushed into a recovery ledger.

Workflow

Action 1 -> Register Voter 1012
Action 2 -> Cast Vote Voter 1012

The system always removes the most recent action first when an undo is triggered.

Responsibilities

- Revert incorrect registrations
- Revert incorrect vote casts
- Maintain data integrity and audit transparency

Data Structure Used

Stack

Reason for Selection

Stacks follow the Last-In-First-Out (LIFO) principle, which perfectly mirrors the mechanical requirements of a system "Undo" feature, allowing immediate reversal of the latest system state change in O(1) time.

---

5. Secure Verification Module

Election systems frequently require the retrieval and verification of voter identities to prevent fraudulent voting. To improve verification efficiency, voter cards are mapped directly to their assigned Voter IDs.

Responsibilities

- Fast voter identity verification

- Reduce administrative delays at polling booths
- Prevent duplicate voting

Data Structure Used

Hash Table

Reason for Selection

Hash Tables allow for secure, direct retrieval of verification data in $O(1)$ average time, crucial for maintaining fast throughput at high-density polling stations.

---

6. Search and Retrieval Module

Election authorities frequently require the retrieval of specific voter records.

Cases are searched systematically using their unique assigned Voter IDs.

Example

Sorted IDs:
1001 1002 1003 1004 1005 1006

Search 1005

Step 1: Mid = 1003

Step 2: Move Right

Step 3: Found 1005

Responsibilities

- Fast voter retrieval
- Reduce search time
- Improve user productivity

Algorithm Used
Binary Search

---

7. Logistics Analytics Module

Election materials must be transported from Election Headquarters to various polling stations. These physical relationships are represented using a Logistics Graph.

Example

HQ - Station 1 (Distance: 12km)
HQ - Station 2 (Distance: 20km)
Station 1 - Station 3 (Distance: 8km)
↓

Meaning: Polling stations and the HQ are connected by physical routes with specific distances.

Responsibilities

- Store physical route relationships
- Visualize network connectivity
- Support material dispatch analytics

Data Structure Used
Undirected Graph
Traversal Technique
Breadth First Search (BFS), Depth First Search (DFS), and Dijkstra's Algorithm

---

8. Station Ranking Module

In election systems, resource allocation must prioritize regions with higher voter density. To manage this, stations are ordered by their registered voter counts.

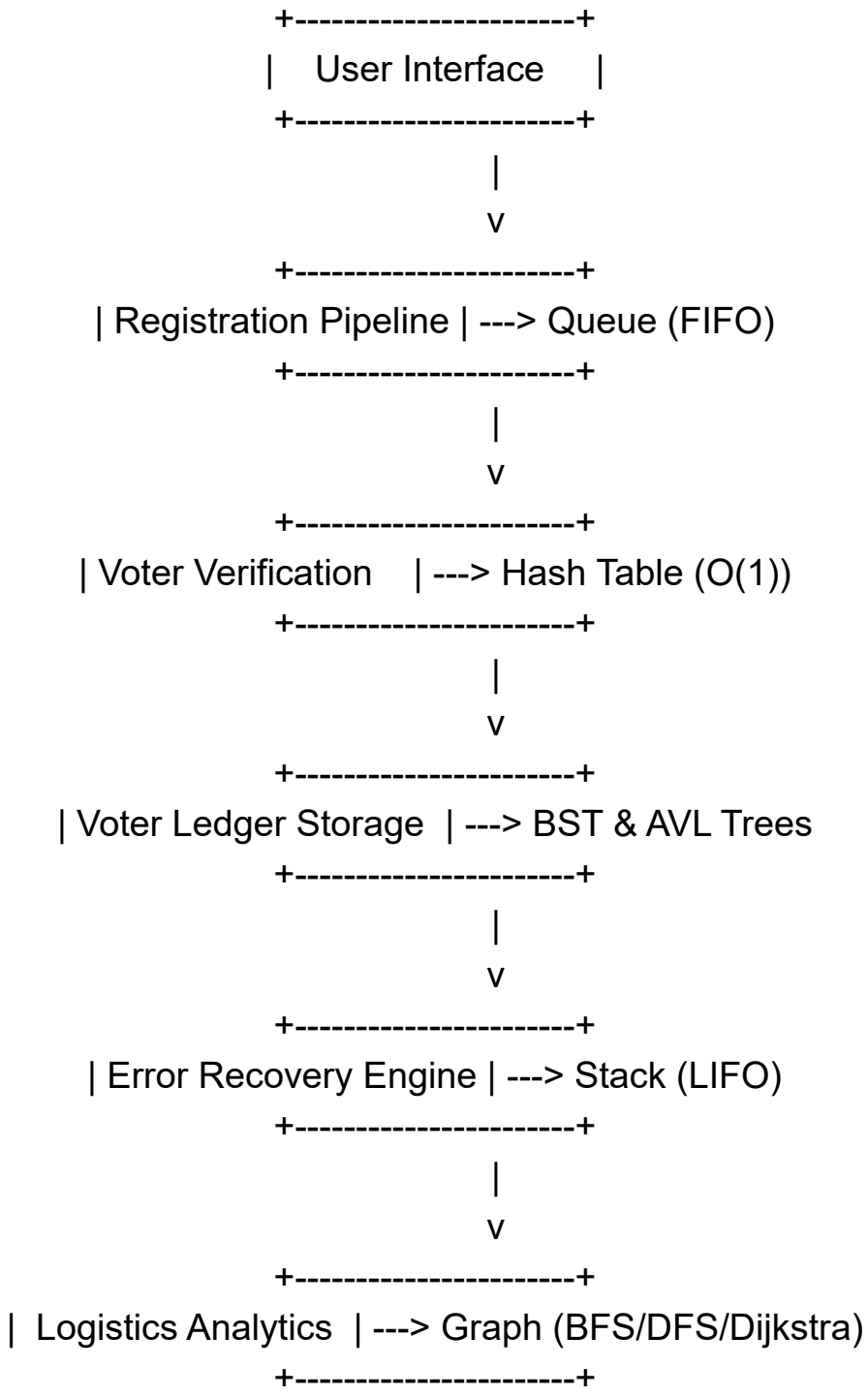
Responsibilities

- Rank polling stations by voter volume
- Optimize resource and personnel allocation
- Support operational decision-making

Algorithm Used

Merge Sort

Complete System Architecture Diagram



Algorithm Description

Algorithm 1: Voter Registration Processing

Objective

Process a new voter registration request from the pending queue.

Steps

- Check whether the registration queue is empty.
- Remove the front request from the queue.
- Validate the target Polling Station ID.
- Create a new Voter object and assign a unique Voter ID.
- Insert the voter into both the BST and AVL tree ledgers.
- Insert the Voter Card No into the Hash Table.
- Push the "REGISTER" action to the recovery Stack.

Pseudocode

```
IF Queue Empty
   Stop
ELSE
   Dequeue Request
   Create Voter
   Insert into BST & AVL
   Insert into HashTable
   Push "REGISTER" to Stack
END IF
```

Complexity

Time Complexity: O(logN)

Space Complexity: O(1)

Algorithm 2: Data Recovery (Undo Operation)

Objective

Revert the last administrative action to correct data entry errors.

Steps

1. Check whether the recovery stack is empty.
2. Pop the top Action object from the stack.
3. If the action was a registration, remove the record from the Hash Table and update station counts.
4. If the action was a vote cast, revert the voter's status and decrement the station's vote count.

Pseudocode

```
IF Stack Empty
 Stop
ELSE
 Pop Action
 IF Action == REGISTER
 Remove from HashTable
 Decrement Station Registered Count
 ELSE IF Action == VOTE
 Set Voter.hasVoted = False
 Decrement Station Votes Cast
 END IF
END IF
```

Complexity

Time Complexity: O(1)

Space Complexity: O(1)

Algorithm 3: Secure Voter Verification

Objective

Verify a voter's identity instantly using their unique Voter Card No to prevent fraud.

Steps

1. Input the Voter Card No string.
2. Calculate the hash index using a polynomial rolling hash function.
3. Access the corresponding bucket in the Hash Table.
4. Traverse the bucket to find the matching card string.
5. Return the associated Voter ID, or return -1 if not found.

Pseudocode

```
INPUT VoterCardNo
Index = HashFunction(VoterCardNo)
FOR EACH pair IN Table[Index]
 IF pair.Card == VoterCardNo
 RETURN pair.VoterID
 END IF
END FOR
RETURN -1
```

Complexity

Time Complexity: O(1) (Average)

Space Complexity: O(1)

Algorithm 4: Station Ranking

Objective

Rank polling stations based on registered voter volume using Merge Sort.

Steps

- Divide the array of polling stations into two halves.
- Recursively sort both halves.
- Merge the sorted halves in descending order based on registered voter counts.
- Repeat until the entire array is sorted.

Complexity

Time Complexity: O(Nlog N)

Space Complexity: O(N)

Algorithm 5:Binary Search for Voter

Objective

Find a voter record efficiently using their Voter ID.

Steps

1. Ensure the voter array is sorted by ID.
2. Select the middle element.
3. Compare the target ID with the middle element.
4. Move the search space to the left or right half based on the comparison.
5. Repeat until found or the search space is exhausted.

Complexity

Time Complexity: O(logN)

Space Complexity: O(1)

Algorithm 6: Logistics Shortest Route

Objective

Find the shortest material dispatch path from Election HQ to all polling stations.

Steps

1. Initialize distances to all nodes as infinity, setting HQ distance to 0.
2. Push the HQ node to a Min-Priority Queue.
3. Extract the minimum distance node from the queue.
4. Update distances to all adjacent neighbor nodes.
5. If a shorter path is found, push the updated distance to the Priority Queue.
6. Repeat until the queue is empty.

Pesudocode

```
Dist[] = INFINITY
Dist[Source] = 0
MinHeap.Push(0, Source)

WHILE MinHeap NOT EMPTY
   U = MinHeap.ExtractMin()
   FOR EACH Neighbor V OF U
      IF Dist[U] + Weight(U, V) < Dist[V]
```

```
 Dist[V] = Dist[U] + Weight(U, V)
 MinHeap.Push(Dist[V], V)
 END IF
 END FOR
END WHILE
```

Complexity

Time Complexity: O((V+E) log V)

Space Complexity: O(V)

Algorithm 7: Logistics Network Traversal

Objective

Trace all connected routes in the election logistics network using Depth First Search (DFS).

Steps

1. Start from the Election HQ node.
2. Mark the node as visited.
3. Visit connected unvisited polling station nodes.
4. Continue recursively.
5. Print the traversal path to visualize connectivity.

Complexity

Time Complexity: O(V + E)

Space Complexity: O(V)

Cycle Detected.

Complexity Analysis

Operation	Algorithm / Data Structure	Time Complexity	Space Complexity
Request Registration	Queue Enqueue	O(1)	O(1)
Process Registration	Dequeue + AVL/Hash Insert	O(log N)	O(1)
Undo Last Action	Stack Pop	O(1)	O(1)
Search Voter	Binary Search	O(log N)	O(1)
Verify Voter Identity	Hash Table Lookup	O(1) avg	O(1)
Rank Polling Stations	Merge Sort	O(N log N)	O(N)
Logistics Search	Breadth First Search (BFS)	O(V+E)	O(V)
Shortest Route	Dijkstra's Algorithm	O((V+E) log V)	O(V)

Where:

- N = Total number of voters / stations
- V = Number of graph nodes (locations)
- E = Number of graph edges (routes)

Screenshots of Execution

```
Loaded 11 voters and 4 stations from file.

╔═══════════════════════════════════════════════════════════════════════╗
║  SMART ELECTION MANAGEMENT & VOTER ANALYTICS SYSTEM                   ║
╠═══════════════════════════════════════════════════════════════════════╣
║  1.  Submit Voter Registration Request   (Queue)                      ║
║  2.  Process Next Registration            (Queue -> BST/AVL/Hash)     ║
║  3.  View Pending Registration Queue                                  ║
║  4.  Undo Last Operation                  (Stack)                     ║
║  5.  Search Voter by ID                   (Linear/Binary Search)      ║
║  6.  Verify & Cast Vote                    (Hashing)                  ║
║  7.  Add New Polling Station                                          ║
║  8.  Rank Polling Stations by Voter Volume (Sorting)                  ║
║  9.  Logistics Network - Add Route                                    ║
║  10. Logistics Network - BFS / DFS / Dijkstra                         ║
║  11. Election Dashboard                                               ║
║  0.  Exit                                                             ║
╚═══════════════════════════════════════════════════════════════════════╝
Enter choice: 1
Enter name: Rahul Sharma
Enter age: 34
Enter gender (M/F/O): M
Enter Voter Card No (unique ID proof): VC10012
Enter target Polling Station ID: 2
Registration request added to queue.

Enter choice: 2
Voter registered successfully. Assigned Voter ID: 1012

Enter choice: 6
Enter Voter Card No: VC10012
Identity verified via hash lookup. Vote cast successfully for Voter ID 1012.

Enter choice: 8
Polling Stations Ranked by Voter Volume (High to Low):
1. Central Park Booth (Central Zone) - Registered: 5 | Capacity: 1000
2. Sector 12 Booth (North Zone) - Registered: 3 | Capacity: 800
3. Riverside Hall (East Zone) - Registered: 3 | Capacity: 600
4. Hillview School (South Zone) - Registered: 1 | Capacity: 500

Enter choice: 11
========== ELECTION DASHBOARD ==========
Total Registered Voters : 12
Total Votes Cast        : 5
Overall Turnout         : 41.7%
Pending Registrations   : 0
Recovery Stack Depth    : 2

Polling Station Overview:
ID   Name               Area          Capacity  Registered  Votes   Turnout%
1    Sector 12 Booth    North Zone    800       3           1       33.3%
2    Riverside Hall     East Zone     600       3           1       33.3%
3    Central Park Booth Central Zone  1000      5           3       60.0%
4    Hillview School    South Zone    500       1           0       0.0%
```

```
================================================================================
||  SMART ELECTION MANAGEMENT & VOTER ANALYTICS SYSTEM                        ||
================================================================================
||  1.  Submit Voter Registration Request   (Queue)                           ||
||  2.  Process Next Registration            (Queue -> BST/AVL/Hash)          ||
||  3.  View Pending Registration Queue                                       ||
||  4.  Undo Last Operation                  (Stack)                          ||
||  5.  Search Voter by ID                   (Linear/Binary Search)           ||
||  6.  Verify & Cast Vote                    (Hashing)                       ||
||  7.  Add New Polling Station                                               ||
||  8.  Rank Polling Stations by Voter Volume (Sorting)                       ||
||  9.  Logistics Network - Add Route                                         ||
||  10. Logistics Network - BFS / DFS / Dijkstra                              ||
||  11. Election Dashboard                                                    ||
||  0.  Exit                                                                  ||
================================================================================
Enter choice: 1
Enter name: Harish Malviya
Enter age: 20
Enter gender (M/F/O): M
Enter Voter Card No (unique ID proof): VC10013
Enter target Polling Station ID: 3
Registration request added to queue.

Enter choice: 1
Enter name: Anita Desai
Enter age: 28
Enter gender (M/F/O): F
Enter Voter Card No (unique ID proof): VC10014
Enter target Polling Station ID: 1
Registration request added to queue.

Enter choice: 3

Pending Registration Requests (FIFO order):
1. Harish Malviya (Card:VC10013) -> Station 3
2. Anita Desai (Card:VC10014) -> Station 1

Enter choice: 2
Voter registered successfully. Assigned Voter ID: 1013
```

```
Enter choice: 5
Enter Voter ID to search: 1013
1. Linear Search  2. Binary Search
Choice: 2
Found in 3 comparisons.
ID: 1013 | Name: Harish Malviya | Age: 20 | Gender: M | Station: 3 | Voted: No

Enter choice: 4
Undo: Registration of Voter ID 1013 reverted.
Note: BST/AVL ledgers retain the entry permanently for audit transparency.

Enter choice: 5
Enter Voter ID to search: 1013
1. Linear Search  2. Binary Search
Choice: 2
Voter not found. (Comparisons made: 4)
```

```
Enter choice: 7
Station name: Robotics Institute Booth
Area / Zone: Tech Park
Capacity: 1200
Polling station added with ID 5.

Enter choice: 9
Available Nodes:
0. Election HQ
1. Sector 12 Booth
2. Riverside Hall
3. Central Park Booth
4. Hillview School
5. Robotics Institute Booth

From node ID: 3
To node ID: 5
Distance (km): 5
Route added successfully.

Enter choice: 10
1. BFS from HQ  2. DFS from HQ  3. Dijkstra Shortest Routes from HQ
Choice: 3
Shortest distance from Election HQ:
Election HQ: 0 km
Sector 12 Booth: 12 km
Riverside Hall: 20 km
Central Park Booth: 20 km
Hillview School: 30 km
Robotics Institute Booth: 25 km

Enter choice: 0
Exiting Election Management System.
```

Findings

- Queue-based processing prevents registration bottlenecks and enforces a fair FIFO policy.
- The Stack effectively implements an administrative failsafe, perfectly reversing complex state changes (vote counts, verifier entries).
- Hashing guarantees near-instant verification, crucial for high-traffic environments like polling stations.
- Merge Sort accurately prioritizes logistics planning by dynamically ranking stations based on load.
- Graph routing (Dijkstra) successfully isolates the mathematical shortest path for secure EVM transport from HQ to designated zones.

Sample Dataset Statistics

The system loads:

- 4 Polling Stations across distinct zones.
- 11 pre-registered demo voters with varied voting statuses.
- Fixed logistical graph routes for immediate route testing.

Conclusion

The Smart Election Management & Voter Analytics System successfully integrates Queues, Stacks, Hash Tables, BSTs, AVLs, Merge Sort, and Graph Traversal algorithms to simulate a modernized election pipeline.
The project proves that integrating fundamental DSA concepts leads to scalable, highly optimized software capable of handling sensitive public administration tasks, reducing human error, and mathematically optimizing resource deployment.

GitHub Repository Link

GitHub Repository:-https://github.com/harish0stack/dsa1-casestudy